EDUGRADE GLOBAL

Sistema Internacional de Calificaciones Multimodelo y Trazabilidad Académica

Integrantes
Mendez Morante | Molinas | Vukusic

Materia
Ingenieria de Datos II

Profesor
Salas, Joaquín

Fecha
23/02/2026

Grupo
1

UADE

PERSISTENCIA POLIGLOTA

Contenido

- Contexto y Objetivo
- Conectividad
- Bases de datos
- Escalabilidad
- Demo del programa

Sistema Nacional Unificado de Calificaciones

El objetivo es unificar calificaciones internacionales y trayectorias complejas en un sistema resiliente con **99.5% de disponibilidad**. Estará diseñado para escalar y procesar eficientemente **más de 1M de registros** para el Ministerio de Educación.

RF1
Registro Académico Oficial

RF2
Conversión y Normalización

RF3
Relaciones Académicas Complejas

RF4 y RF5
Análisis, reportes y auditoría

Métricas Críticas de Conectividad

Latencia
Menor a 10ms para operaciones CRUD en MongoDB y Redis, y entre 10-50ms para recorridos de grafos en Neo4j.

Tasa de Errores
Mantener un Error Rate inferior al 1% para asegurar la confiabilidad de las conversiones y registros.

Pool de Conexiones
Monitorear que la saturación del pool no supere el 80% para evitar errores de conexión agotada.

Throughput (TPS)
> 1000 transacciones por segundo especialmente en Cassandra

Plan de Benchmarking

MongoDB: mongoperf para evaluar la latencia de consulta de las operaciones CRUD

Neo4j: neo4j-benchmark para medir rendimiento del traversal

Cassandra: cassandra-stress para validar la escalabilidad de las escrituras de auditoría e inmutabilidad.

Redis: redis-benchmark para medir latencia y throughput de las reglas de conversión cacheadas.

Global: Pruebas de estrés con Artillery o JMeter simulando el flujo completo

Troubleshooting

BDD	Cuellos de Botella	Soluciones
MongoDB	Alta latencia en búsquedas por nombre o institución. Agotamiento del pool de conexiones al registrar masivamente.	Crear índices para consultas frecuentes. Aumentar maxPoolSize y asegurar el cierre de conexiones en el backend. Sharding.
Neo4j	Lentitud traversales (ej: buscar equivalencias indirectas).	Implementar LIMIT en búsquedas de grafos
Cassandra	Lecturas analíticas pesadas sobre grandes volúmenes históricos.	Diseñar mejor el esquema orientado a la consulta y usar clustering keys
Redis	Errores de memoria	Implementar TTL (Time To Live) en datos temporales

MongoDB - RF1 (Registro Académico Oficial)

Motor documental (JSON/BSON) para centralizar la información principal.

ESTUDIANTES: Perfiles heterogéneos con múltiples trayectorias y datos de contacto variables según el país.

INSTITUCIONES: Atributos específicos de cada sistema educativo sin forzar una estructura única.

MATERIAS: Coexistencia de diferentes métodos de evaluación (créditos, módulos, finales).

- **Esquema Flexible:** Permite almacenar datos semi-estructurados sin una definición rígida previa.
- **Escalabilidad Horizontal:** Diseñado para crecer mediante sharding, facilitando el manejo de grandes volúmenes de datos.
- **Alta Velocidad de Lectura/Escritura:** Ideal para registros académicos que requieren consultas frecuentes y actualizaciones rápidas.

MongoDB - RF1 (Registro Académico Oficial)

Referencia de InstitucionID (vs. Embebido)

Garantizaríamos la consistencia de los datos oficiales y optimizaríamos el almacenamiento ante el volumen de 1M de registros, evitando la duplicación innecesaria de información institucional.

Motivo	Ventaja Técnica
Evitar Redundancia	**No se repite la información de la institución en cada materia**
Consistencia de Datos	**Si la institución cambia de nombre, se actualiza en un solo lugar**
Límite de Documento	**Evita superar los 16MB de MongoDB al no agrupar miles de materias en una institución**

```
{
  _id: ObjectId('6998e3539c72ed2780138d1f'),
  nombre: 'Física I (AR)',
  nivel: 'Básico',
  institucion: ObjectId('6998e3539c72ed2780138ce3'),
  metadata: {
    creditos: 4,
    modalidad: 'Virtual',
    cupo_maximo: 40,
    requiere_laboratorio: 'Sí'
  }
},
{
  _id: ObjectId('6998e3539c72ed2780138d22'),
  nombre: 'Física II (AR)',
  nivel: 'Básico',
  institucion: ObjectId('6998e3539c72ed2780138ce3'),
  metadata: { creditos: 4, requiere_laboratorio: 'Sí' }
},
{
  _id: ObjectId('6998e3539c72ed2780138d25'),
  nombre: 'Programación I (AR)',
  nivel: 'Avanzado',
  institucion: ObjectId('6998e3539c72ed2780138ce3'),
  metadata: {
    creditos: 6,
    modalidad: 'Virtual',
    cupo_maximo: 83,
    requiere_laboratorio: 'Sí'
  }
},
```

Neo4j

Rol

Representar relaciones académicas y realizar consultas de equivalencia y trazabilidad.

Eficiencia

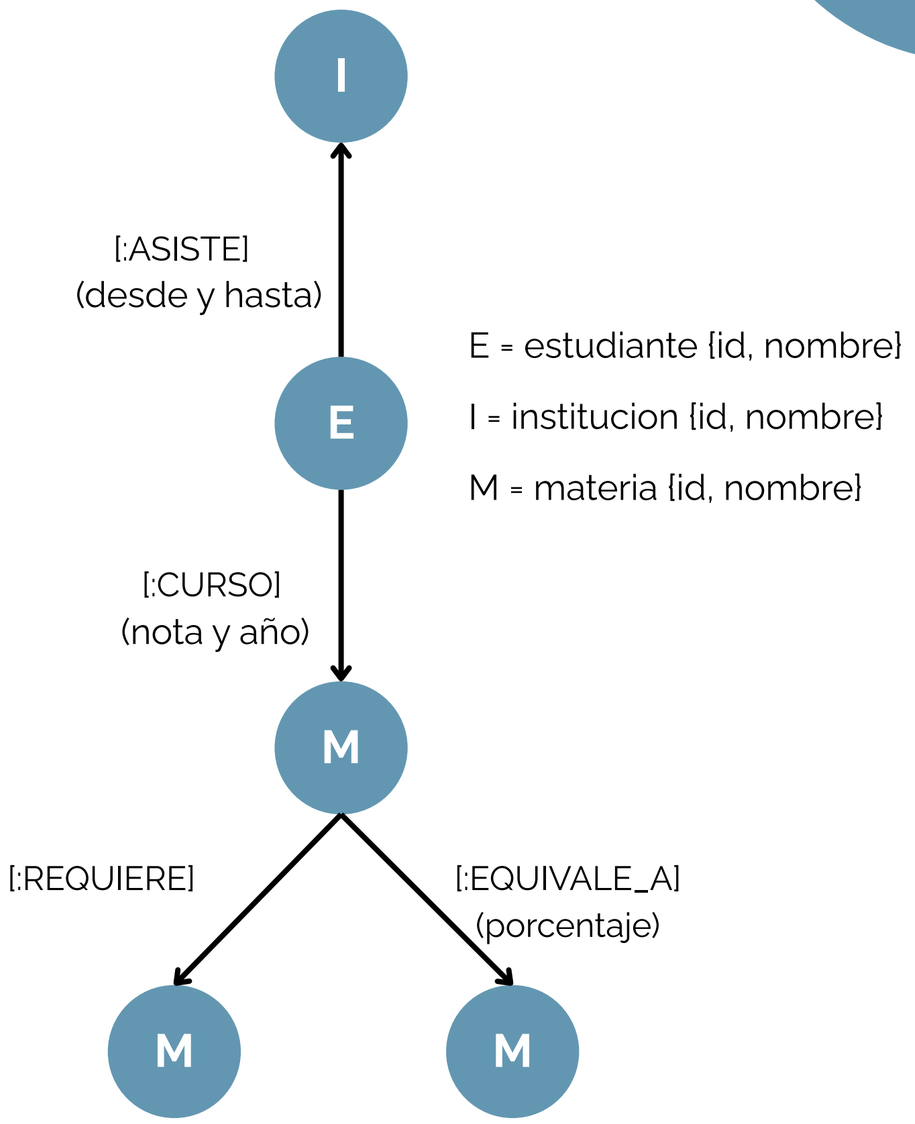
Consultas relacionales complejas se resuelven con saltos directos; evita JOINs costosos en SQL

Trazabilidad

Rutas de carrera, equivalencias y dependencias representadas nativamente para respuesta rápida.

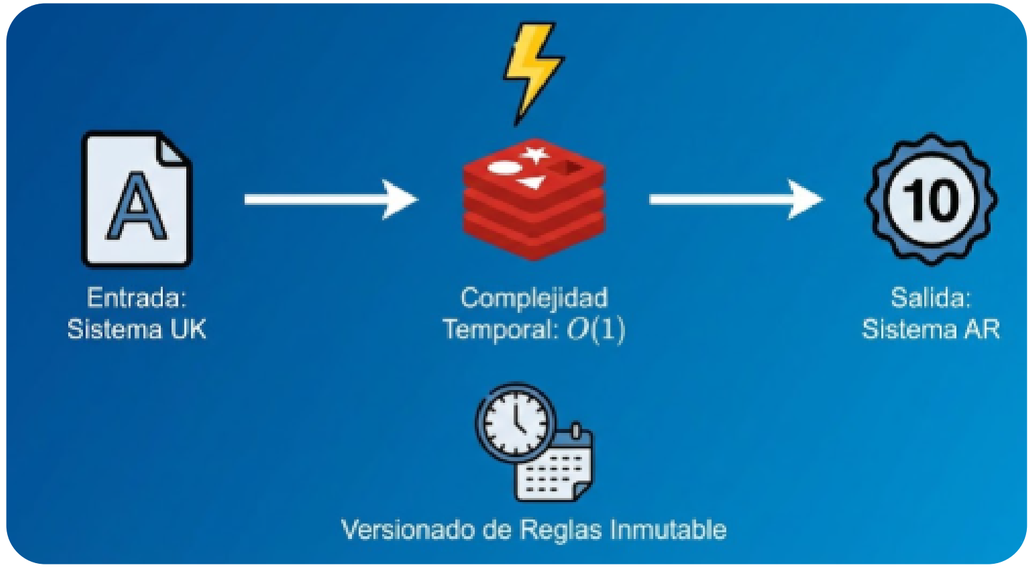
Index-Free Adjacency

Los nodos tienen punteros físicos en memoria hacia sus relaciones. Por lo que saltar de un Estudiante a su Materia cuesta O(1)



Redis

Es el motor de cálculo en tiempo real. Actúa como el "traductor oficial" del Ministerio, permitiendo que una calificación de cualquier país se normalice instantáneamente bajo las reglas vigentes.



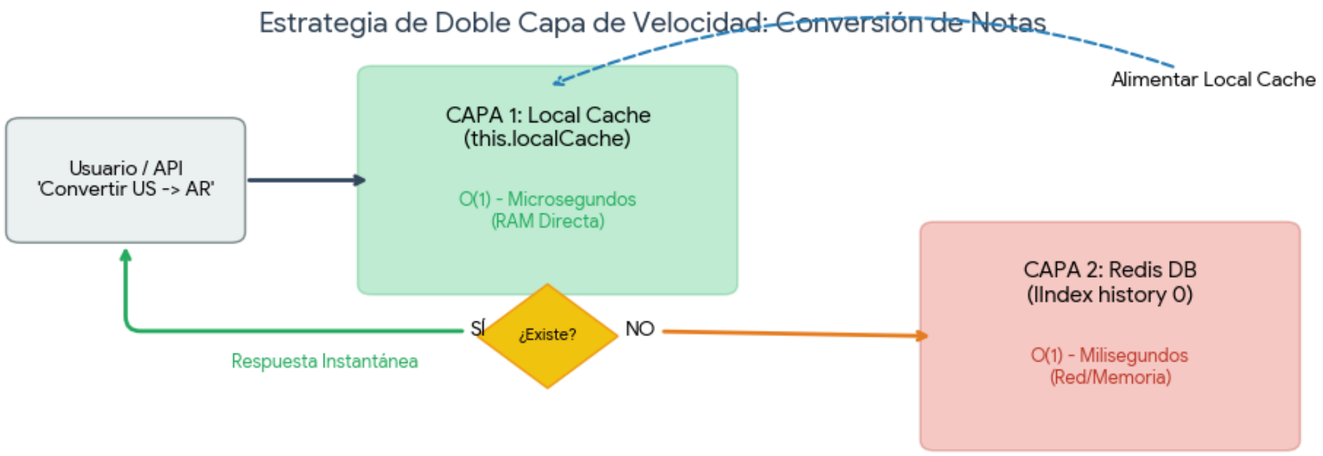
Ventajas

- Latencia O(1): Al operar 100% en memoria, la búsqueda de normativas es inmediata, sin importar cuántos miles de alumnos estén operando el sistema.
- Versionado de Leyes: Utilizamos estructuras de Redis Lists (LPUSH) para almacenar el historial. Esto permite que el Ministerio cambie una escala hoy, pero mantenga el registro de cómo se calculaban las notas ayer.

Capas de velocidad: Redis y Cache local

Doble capa de velocidad : CACHE LOCAL Y REDIS

this.localCache	Redis
Si una conversión se pide muchas veces, el servidor ni siquiera le pregunta a Redis; lo tiene en su propia memoria RAM.	Si no está en el caché local, va a Redis



BENEFICIOS:
1. Alta Disponibilidad: Si Redis cae, el Local Cache sigue sirviendo lo ya conocido.
2. Escalabilidad: Redis centraliza las reglas para múltiples instancias de la App.
3. Velocidad: Se evita el acceso a Disco (Mongo) o cálculos pesados en cada petición.

Implementación:

- **Nivel aplicacion:** se crea un historial donde nunca borras, solo agregas (Append) nuevas versiones de las normativas. timestamp en regla.js y Lpush en repositorioreglas.js
- **Nivel BDD:** Redis toma ese lPush y lo guarda en su archivo AOF.

Cassandra - RF4 y RF5 (Reportes y Auditoría)

Alta Disponibilidad

Arquitectura masterless sin SPOF

Escrituras Ultra-rápida

Optimizada para flujos de datos continuos (Append-only)

Query-Driven Design

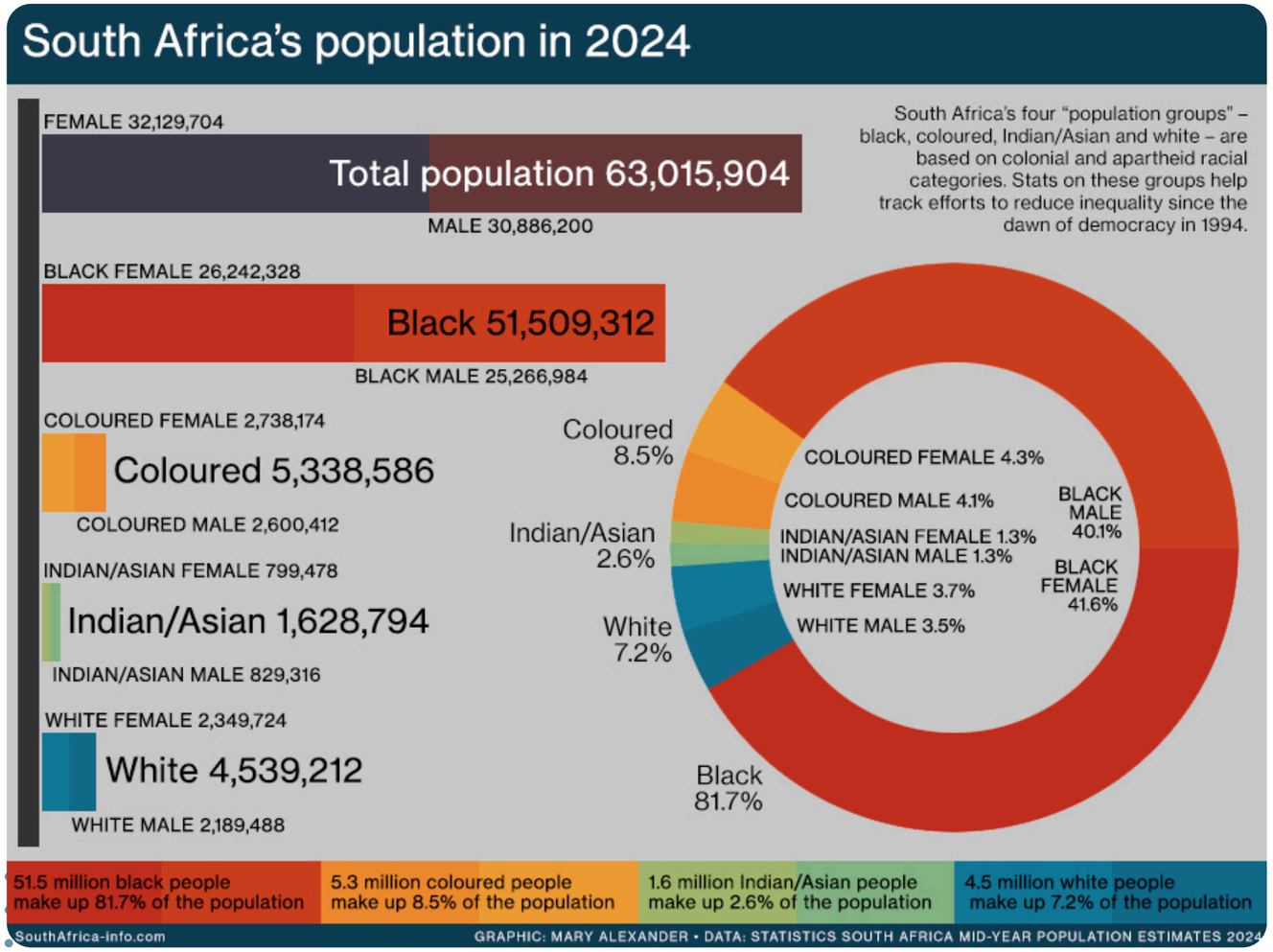
Esquema diseñado específicamente para las consultas necesarias

Escalabilidad

Capacidad de procesar millones de registros añadiendo nodos al clúster

Tabla	Partition Key	Objetivo
analitica_por_institucion	institucion_id	RF4: Reportes por institución. Promedios y aprobación por materia/año.
registro_auditoria	(tipo_entidad, anio_mes)	RF5: Auditoría y trazabilidad. Trazabilidad inmutable y cronológica de eventos.
metricas_desvio_estandar	contexto_id	RF4: Detección de desvíos. Cálculo de dispersión y desvíos estadísticos.

Teorema CAP y Escalabilidad



BDD	Teorema CAP	Clustering y Nodos	Replicación
MongoDB	CP. Se prioriza que la información sea exacta y consistente en el nodo primario antes de ser leída.	N3 W2 R2	Replica Set con failover automático
Neo4j	Balance CP/AP	Core Servers: 3 nodos Replicas Servers: 9 nodos	Replicación asíncrona hacia las Replicas Servers provinciales.
Cassandra	AP. Para el registro masivo de auditoría (append-only) donde el sistema nunca debe dejar de recibir datos.	N3 W1 R1 Consistency level: LOCAL_ONE	NetworkTopologyStrategy DC1: 3, DC2: 3, DC3: 3
Redis	AP. Priorizamos la baja latencia para que el proceso de conversión de escalas sea instantáneo para el usuario.		Replicación asíncrona con mecanismo de persistencia AOF

GRACIAS POR SU ATENCIÓN

Grupo 1

Repositorio del sistema desarrollado

https://github.com/AgusVuku7/TPO-BDD2.git

UADE